Order reduction by co-contraction

A complete, fully-derived theory guide for `order-reduction-sim`

Working document for the Royal Society URF proposal
Repository: `order-reduction-sim/report/theory.tex`

September 2026

How to read this document (interactive)

This PDF is **hyperlinked**. Click any blue section reference or any entry in the table of contents to jump there. Every equation number is also clickable and jumps back to where it was first written.

What is new in this version. Nothing is left as "it can be shown that". Every equation is derived from the line before it, with the algebra written out. Every numerical result has a fully worked example with the specific numbers from the code. The two results PDFs (`bias_law.pdf`, `mechanism.pdf`, `learner2.pdf`) are each explained panel by panel.

Suggested path for a first read:

1. §1 — the cup-and-arm story in one page.
2. §3–§4 — full derivation of $\rho(\xi)$, step by step.
3. §7 — the bias law, derived term by term (the main finding).
4. §8.5 — what Learner 1's figure shows, panel by panel.
5. §9 — Learner 2, with EKF and ARD derived from scratch.
6. §9.8 — what Learner 2's figure shows, panel by panel.
7. §11 — what we conclude and what we do *not* claim.

Green boxes are confirmed results. Red boxes are refuted claims or limits. Yellow boxes are everyday examples. Grey boxes are line-by-line derivations. White boxes explain a technical method (EKF, ARD, gradient descent) from first principles.

Contents

1 The story in one page

You hold a cup of coffee and learn to move it without spilling. The cup has its own physics (weight, wobble, slosh). Your arm has physics too (stiffness, damping). Together they form one **coupled system**.

Thrishanth Nanayakkara's idea, developed after the July 2026 meeting, is:

> When the coupled system is **high order** (many ways to wobble), the brain may **stiffen the arm** first. Stiffening pushes the fast wobbles so far into the past that, during one reach, you mostly feel the **slow, dominant** response. You learn that slow part safely. Then you **relax** and learn the fast parts on top.

The simulation in `order-reduction-sim` tests this idea with two different computer learners. Both use the same cup and the same five tuning schedules; they differ in *how* they learn, and that difference turns out to matter enormously for whether the idea can even be tested.

Lay example: Cup of coffee

Soft arm, fast reach: the coffee sloshes, the cup wobbles, you feel many things at once. If your mental model is wrong, the movement fails (spill).
Stiff arm, slow reach: you squeeze hard. Slosh dies in milliseconds. During your 0.8 s reach you mostly feel "how heavy/sluggish is this cup?" — one main lag. Safer, simpler, but you cannot yet learn the slosh.
Then relax: once you know the main lag, you soften the arm and move faster. The slosh reappears in what you feel. Now you can learn it — but only if your coarse model is already good enough that the reach does not fail.

Plain-language guide: Why two learners? (read this before anything else)

The first attempt at simulating this idea ("Learner 1") used a computer program that *always knew* the cup had three separate wobbles, it just had to find the right numbers for them. That program could not show the central claim, no matter how it was tuned, because assuming the right answer in advance means the only remaining error is random noise, not the *systematic* mistake Thrish is talking about. §8 works through why this happened in full. §9 builds a second program ("Learner 2") that does *not* assume the answer, and that program does show the effect. Both are kept in the repository and in this report, because the failure of Learner 1 is itself informative.

2 Complete notation table

Every symbol used in this document and in the code is listed here.

Symbol	Units	Meaning
$x(t)$	m	Position of the coupled hand–object along one axis.
$\dot{x}$, $\ddot{x}$	m/s, m/s^2	First and second time-derivatives of x: velocity and acceleration.
$F(t)$	N	Force applied (your muscle command).
M	kg	Effective inertia (mass of arm + object).
B	N s/m	Damping (friction + muscle viscosity). *You increase this by co-contracting.*
K	N/m	Stiffness (muscle tone + object stiffness). *You increase this too.*
ξ	—	**Damping ratio** (co-contraction level in the simulation).
ω_n	rad/s	Natural frequency, $\sqrt{K/M}$.
p_d	1/s	**Dominant (slow) pole** — how sluggish the system feels.
p_f	1/s	**Fast (transient) pole** — how quickly wobbles die.
$\rho(\xi)$	—	**Separation ratio** $\lvert p_f\rvert/\lvert p_d\rvert$ when $\xi > 1$. Grows rapidly with stiffness.

Symbol	Units	Meaning
s	1/s	Laplace variable (frequency domain).
$G(s)$	—	Transfer function: output per unit input in frequency domain.
τ_d, τ_1, τ_2	s	**Time constants** of the three-lag object model (slow, medium, fast).
$\hat{\tau}_d, \hat{\tau}_1, \hat{\tau}_2$	s	Learner's **estimates** of those time constants.
a	—	Discrete-time decay factor $e^{-\Delta t/\tau}$ for one lag.
$u(t)$	—	Command signal sent to the limb (reference reach + exploration).
$y(t)$	—	Felt response (position or force trace).
$\hat{y}(t)$	—	Learner's **prediction** of what it should have felt.
Δt or Δt	s	Sample interval (0.01 s in code).
t_{move}	s	Duration of one reach segment (controls movement speed).
σ	—	Sensor / output noise standard deviation.
surprise	—	$\max_t \lvert y(t) - \hat{y}(t)\rvert$; largest prediction mismatch in a trial.
S_BASE	—	Failure threshold scale (0.15 in code).
n_{unlocked}	—	How many time constants Learner 1 is allowed to fit (1, 2, or 3).
w_1, w_2	—	**Participation weights** (Learner 2): how active each transient mode is (0–1).
n_{eff}	—	**Effective order** $= 1 + w_1 + w_2$ (Learner 2).
q	—	State vector in log-parameters + weights (Learner 2).
P	—	Covariance matrix: how uncertain the learner is about q.
Q	—	Process noise covariance: how much beliefs are allowed to drift per trial.
R	—	Measurement noise variance used in the filter update.
H or J	—	Jacobian: sensitivity of prediction to each parameter, $\partial\hat{y}/\partial q$.
α_i	—	ARD precision (penalty strength) on weight w_i.
γ_i	—	"How well determined" weight i is, between 0 and 1.
$\kappa(\Phi)$	—	Condition number of the identification matrix (hard vs easy to learn).
λ_i	—	Eigenvalue of a curvature/information matrix, i.e. how steep the cost function is in one direction.
$e(t)$	—	Prediction error, $y(t) - \hat{y}(t)$.
$\mathcal{C}$	—	Cost function (sum of squared errors) that a learner minimises.
η	—	Learning rate (step size) in gradient descent.

3 Part I — The second-order mass–spring–damper

3.1 The physical equation

Along one movement axis, the coupled arm and object are modelled as a **mass** pulled by a **spring** and slowed by a **dashpot** (damper). This is one of the oldest equations in mechanics, so we derive every consequence of it from scratch.

$$\boxed{M\ddot{x} + B\dot{x} + Kx = F(t)} \tag{1}$$

- $M\ddot{x}$ — Newton's second law: force equals mass times acceleration. This term resists *changes in velocity* ("hard to get moving, hard to stop").
- $B\dot{x}$ — a damper produces a force proportional to velocity and opposing it. This term resists *velocity itself* ("motion is slowed, wobbles die out").
- Kx — a spring produces a restoring force proportional to displacement (Hooke's law). This term *pulls back toward rest* ("rubber band").
- $F(t)$ — the external force you apply (your muscle command), which can change with time.

Lay example: Mass–spring–damper

Push a shopping trolley. M is the trolley mass: heavy trolleys resist starting and stopping. K is how stiffly you hold the handle (and any elasticity in what you are carrying): a stiff hold snaps

back quickly if disturbed. B is friction in the wheels plus how much you resist the handle moving: high friction damps out any wobble. Co-contraction in the arm increases both B and K together — you grip tighter (raising K) and resist motion more (raising B).

3.2 Laplace transform and transfer function

We now convert (1), a differential equation in time, into an algebraic equation in a new variable s. This is the single most useful trick in linear systems theory, because differentiation becomes multiplication.

Step-by-step: Deriving the transfer function

Step 1. The Laplace transform of a function $f(t)$ is $F(s) = \int_0^\infty f(t)e^{-st}\mathrm{d}t$. Two standard properties we use:

$$\mathcal{L}\{\dot{x}\} = sX(s) - x(0), \qquad \mathcal{L}\{\ddot{x}\} = s^2X(s) - sx(0) - \dot{x}(0).$$

Step 2. Assume the system starts at rest: $x(0) = 0,\ \dot{x}(0) = 0$ ("zero initial conditions"). Then the derivative rules simplify to $\mathcal{L}\{\dot{x}\} = sX(s)$ and $\mathcal{L}\{\ddot{x}\} = s^2X(s)$.
Step 3. Apply the Laplace transform to every term of (1):

$$M\big(s^2X(s)\big) + B\big(sX(s)\big) + KX(s) = F(s).$$

Step 4. Every term on the left contains $X(s)$, so factor it out:

$$\big(Ms^2 + Bs + K\big)X(s) = F(s).$$

Step 5. Divide both sides by $\big(Ms^2 + Bs + K\big)$ and by $F(s)$ to isolate the ratio of output to input:

$$G(s) \;=\; \frac{X(s)}{F(s)} \;=\; \frac{1}{Ms^2 + Bs + K}. \tag{2}$$

This $G(s)$ is the **transfer function**: it tells you how strongly, and with what delay, each frequency component of the force F appears in the position x. Setting $s = j\omega$ (imaginary axis) gives the ordinary frequency response; keeping s general is what lets us find the poles below.

3.3 Natural frequency and damping ratio

The denominator $Ms^2 + Bs + K$ is a generic quadratic in s. It is standard practice to rewrite its coefficients in a form that separates "how fast" from "how oscillatory". Divide the denominator by M:

$$s^2 + \frac{B}{M}s + \frac{K}{M} = s^2 + 2\xi\omega_n s + \omega_n^2,$$

where the right-hand side is the standard second-order form, and matching coefficients gives:

$$\omega_n = \sqrt{\frac{K}{M}} \qquad \text{(natural frequency, rad/s)} \tag{3}$$

$$\xi = \frac{B}{2\sqrt{MK}} \qquad \text{(damping ratio, dimensionless)} \tag{4}$$

Step-by-step: Checking (4) against the matching

From $2\xi\omega_n = B/M$ and $\omega_n = \sqrt{K/M}$:

$$\xi = \frac{B}{2M\omega_n} = \frac{B}{2M\sqrt{K/M}} = \frac{B}{2\sqrt{M^2 \cdot K/M}} = \frac{B}{2\sqrt{MK}},$$

which is exactly (4).

Lay example: What ξ means

- $\xi < 1$: underdamped — the system oscillates (overshoots and rings) before settling, like a bouncy spring.
- $\xi = 1$: critically damped — returns to rest as fast as possible without overshoot, like a well-tuned door closer.
- $\xi > 1$: overdamped — no oscillation at all; the response is a sum of two plain exponential decays, like a door closer with too much oil in it.

Co-contraction in our simulation is mapped to **raising ξ past 1**, i.e. pushing the coupled arm-object system from "bouncy" toward "sluggish but non-oscillatory".

3.4 The two poles

The values of s that make the denominator of (2) zero are called the **poles** of the system. They are the roots of $Ms^2 + Bs + K = 0$, or equivalently of $s^2 + 2\xi\omega_n s + \omega_n^2 = 0$.

Step-by-step: Solving the quadratic for the poles

Step 1. Apply the quadratic formula $s = \dfrac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ to $s^2 + 2\xi\omega_n s + \omega_n^2 = 0$ (here $a = 1$, $b = 2\xi\omega_n$, $c = \omega_n^2$):

$$s = \frac{-2\xi\omega_n \pm \sqrt{(2\xi\omega_n)^2 - 4\omega_n^2}}{2}.$$

Step 2. Simplify inside the square root:

$$(2\xi\omega_n)^2 - 4\omega_n^2 = 4\omega_n^2\xi^2 - 4\omega_n^2 = 4\omega_n^2(\xi^2 - 1).$$

Step 3. Substitute back and take the square root:

$$s = \frac{-2\xi\omega_n \pm 2\omega_n\sqrt{\xi^2 - 1}}{2} = -\xi\omega_n \pm \omega_n\sqrt{\xi^2 - 1}.$$

Step 4. Factor out $-\omega_n$:

$$\boxed{p_\pm = -\omega_n\left(\xi \mp \sqrt{\xi^2 - 1}\right)} \tag{5}$$

(the sign convention here matches the code, with p_+ the pole closer to zero, i.e. the slow one, when $\xi > 1$).

For $\xi > 1$ the quantity under the square root, $\xi^2 - 1$, is positive, so both roots are **real** and negative (no oscillation, matching the "overdamped" description above). For $\xi < 1$, $\xi^2 - 1$ is negative, the square root is imaginary, and the two roots are a complex-conjugate pair (oscillatory).

3.5 The overdamped limit: two separated real poles

For ξ noticeably larger than 1, one pole is much closer to the origin than the other. To see this, expand the square root for large ξ using the binomial approximation $\sqrt{\xi^2 - 1} = \xi\sqrt{1 - 1/\xi^2} \approx \xi(1 - \frac{1}{2\xi^2})$:

Step-by-step: Finding the approximate dominant and fast poles

Step 1. Write $p_+ = -\omega_n(\xi - \sqrt{\xi^2 - 1})$ (the "−" choice in (5), giving the smaller-magnitude pole) and substitute the approximation:

$$p_+ \approx -\omega_n\Big(\xi - \xi + \frac{1}{2\xi}\Big) = -\frac{\omega_n}{2\xi}.$$

Step 2. Substitute $\omega_n = \sqrt{K/M}$ and $\xi = B/(2\sqrt{MK})$:

$$p_+ \approx -\frac{\sqrt{K/M}}{2 \cdot B/(2\sqrt{MK})} = -\frac{\sqrt{K/M} \cdot \sqrt{MK}}{B} = -\frac{K}{B}.$$

This is the **dominant pole**, $p_d \approx -K/B$.
Step 3. Similarly $p_- = -\omega_n(\xi + \sqrt{\xi^2 - 1}) \approx -2\omega_n\xi$, and substituting the same way gives $p_- \approx -B/M$. This is the **fast pole**, $p_f \approx -B/M$.

$$p_d \approx -\frac{K}{B} \qquad \text{(dominant, } \textbf{slow} \text{ pole)} \tag{6}$$

$$p_f \approx -\frac{B}{M} \qquad \text{(fast, } \textbf{transient} \text{ pole)} \tag{7}$$

Lay example: Two poles = two time scales

An impulse response of a system with two real poles is a sum of two decaying exponentials:

$$x(t) \approx A_d e^{p_d t} + A_f e^{p_f t}.$$

Because $|p_f| \gg |p_d|$ in the overdamped, stiffened limit, the fast term $e^{p_f t}$ decays to nearly zero within a few milliseconds, while the slow term $e^{p_d t}$ decays over roughly a second. That slow term is essentially all that remains by the time you notice anything, so it is what you **feel** during a normal reach.

4 Part II — Time-scale separation and $\rho(\xi)$

4.1 Separation ratio, defined and derived exactly

Define the **separation ratio** as how many times faster the fast pole is than the slow pole:

$$\rho(\xi) = \frac{|p_f|}{|p_d|}. \tag{8}$$

Unlike (6)–(7), which were approximations for large ξ, the code (`plant.rho`) uses the **exact** ratio of the two poles in (5). We now derive that exact closed form.

Step-by-step: Deriving the exact $\rho(\xi)$

Step 1. From (5), the two poles are $p_+ = -\omega_n(\xi - \sqrt{\xi^2 - 1})$ and $p_- = -\omega_n(\xi + \sqrt{\xi^2 - 1})$, with $|p_-| > |p_+|$ for $\xi > 1$ (check: at $\xi = 2$, $\xi - \sqrt{3} \approx 0.27$ and $\xi + \sqrt{3} \approx 3.73$, so indeed $|p_-| > |p_+|$). So $p_- = p_f$ (fast) and $p_+ = p_d$ (dominant).
Step 2. Form the ratio, and note ω_n cancels because it multiplies both poles:

$$\rho(\xi) = \frac{|p_f|}{|p_d|} = \frac{\xi + \sqrt{\xi^2 - 1}}{\xi - \sqrt{\xi^2 - 1}}.$$

Step 3. Rationalise by multiplying top and bottom by the conjugate $\xi + \sqrt{\xi^2 - 1}$:

$$\rho(\xi) = \frac{\left(\xi + \sqrt{\xi^2 - 1}\right)^2}{(\xi - \sqrt{\xi^2 - 1})(\xi + \sqrt{\xi^2 - 1})} = \frac{\left(\xi + \sqrt{\xi^2 - 1}\right)^2}{\xi^2 - (\xi^2 - 1)} = \frac{\left(\xi + \sqrt{\xi^2 - 1}\right)^2}{1}.$$

Step 4. Therefore

$$\boxed{\rho(\xi) = \left(\xi + \sqrt{\xi^2 - 1}\right)^2 \quad \text{for } \xi > 1, \qquad \rho(\xi) = 1 \text{ for } \xi \le 1} \tag{9}$$

(for $\xi \le 1$ the poles are not separable into "fast" and "slow" real poles at all, so the code simply defines $\rho = 1$, meaning no separation.)

Lay example: Numbers for $\rho(\xi)$, computed by hand

At $\xi = 2$: $\sqrt{\xi^2 - 1} = \sqrt{3} \approx 1.732$, so $\rho = (2 + 1.732)^2 = 3.732^2 \approx 13.93$, matching the table below.
At $\xi = 3$: $\sqrt{9 - 1} = \sqrt{8} \approx 2.828$, so $\rho = (3 + 2.828)^2 = 5.828^2 \approx 33.97$.

ξ	$\rho(\xi)$	Plain English
0.7	1.0	Soft: no separation ($\xi \le 1$, code sets $\rho = 1$).
1.2	3.47	Fast mode about 3.5× faster than slow.
2.0	13.93	Strong separation.
3.0	33.97	Very stiff: transients finish in ∼1/34 of the slow time.
4.0	61.98	Extreme stiffening.

4.2 Singular perturbation: the reduced first-order plant

When $\rho \gg 1$, the fast mode settles in a time $\tau_f = 1/|p_f| \approx M/B$ that is negligible compared with the movement duration T. The formal tool for discarding a fast mode this way is called **singular perturbation**.

Step-by-step: From two states to one: the singular perturbation argument

Step 1. Write (1) as two coupled first-order equations by introducing velocity $v = \dot{x}$:

$$\dot{x} = v, \qquad M\dot{v} = F - Bv - Kx.$$

Step 2. Introduce the small parameter $\varepsilon = M/(BT) \ll 1$ (inertia is negligible compared to damping over the movement time T). Rescale time so that the fast dynamics of v appear as $\varepsilon\dot{v} = \ldots$
Step 3. The standard singular-perturbation result is: set $\varepsilon \to 0$ in the fast equation and solve it for its **quasi-steady state**, i.e. assume the fast variable v has already settled to whatever value makes its own derivative zero at every instant:

$$0 \approx F - Bv - Kx \quad \Longrightarrow \quad v \approx \frac{F - Kx}{B}.$$

Step 4. Substitute this quasi-steady v back into $\dot{x} = v$:

$$\dot{x} = \frac{F - Kx}{B} \quad \Longrightarrow \quad B\dot{x} + Kx = F \quad \Longrightarrow \quad \frac{B}{K}\dot{x} + x = \frac{F}{K}.$$

Step 5. Identify $\tau_d = B/K$ (matching (6), since $p_d \approx -K/B = -1/\tau_d$):

$$\boxed{\tau_d\dot{x} + x = \frac{1}{K}F, \qquad \tau_d = \frac{B}{K}} \tag{10}$$

Lay example: Reduced model

While very stiff, the coupled system *behaves during one reach* like a single lag τ_d: push once, and the response ramps up with time constant τ_d, exactly as if the object had no wobble at all. The fast wobble happened already, within the first few milliseconds — you did not see it. The cup did not change; your arm **filtered** the experience.

5 Part III — The third-order object in the simulation

The code does not simulate (1) and a separate object on every time step, coupling them together each instant. Instead it uses a **phenomenological stand-in**: a fixed object with three lags, whose fast lags are compressed by $\rho(\xi)$ exactly as the singular-perturbation argument above says they should be. This captures the *outcome* of coupling without re-deriving it from M, B, K on every step, and it lets the object have *two* transient modes rather than one, which is what Thrish specifically asked the simulation to include.

5.1 Object transfer function

The **object alone** (the cup, independent of your arm) is modelled as three first-order lags multiplied together (equivalently: connected in series, so the output of one feeds the input of the next):

$$\boxed{G(s) = \frac{1}{(1+\tau_d s)(1+\tau_1 s)(1+\tau_2 s)}} \tag{11}$$

Each factor $1/(1+\tau s)$ is a single first-order lag with pole at $s = -1/\tau$ (set $1+\tau s = 0 \Rightarrow s = -1/\tau$). So (11) has three poles, at $s = -1/\tau_d$, $-1/\tau_1$, $-1/\tau_2$.

True values in the repository (`TAU_TRUE`):

$$\tau_d = 1.0\,\text{s}, \quad \tau_1 = 0.1\,\text{s}, \quad \tau_2 = 0.05\,\text{s} \quad \Longrightarrow \quad \text{poles at } s = -1,\ -10,\ -20 \text{ rad/s}.$$

Lay example: Three lags in a row

Think of three filters connected end to end:

1. Slow filter ($\tau_d = 1\,\text{s}$): the main heaviness of the cup.
2. Medium filter ($\tau_1 = 0.1\,\text{s}$): a first, faster wobble.
3. Fast filter ($\tau_2 = 0.05\,\text{s}$): a second, even faster wobble.

Your command passes through all three filters one after another; what you feel is the output of the whole chain.

5.2 What co-contraction changes

Co-contraction ξ does **not** change the object (11) itself — the cup's true physics never change in the simulation. It changes what you **experience**, because your own arm filters the signal before it reaches your brain, exactly as (10) showed for the dominant lag. In the simulation this filtering is applied to *both* transient lags:

$$(\tau_d,\ \tau_1,\ \tau_2)_{\text{felt}} = \left(\tau_d,\ \frac{\tau_1}{\rho(\xi)},\ \frac{\tau_2}{\rho(\xi)}\right). \tag{12}$$

Only the two fast lags are divided by $\rho(\xi)$; the dominant lag τ_d is left alone, because (10) shows the dominant mode is the one that *survives* stiffening, not the one that is compressed by it.

Lay example: Numeric check of (12)

At $\xi = 3$, $\rho \approx 34.0$ (computed above). So:

$$\tau_{1,\text{felt}} = \frac{0.1}{34.0} \approx 0.0029\,\text{s}, \qquad \tau_{2,\text{felt}} = \frac{0.05}{34.0} \approx 0.0015\,\text{s}.$$

During a 0.8 s reach, a lag of 0.0029 s finishes in well under 1% of the movement — effectively instantly. You feel only $\tau_d = 1$ s.

5.3 Discrete-time simulation

Computers cannot integrate a continuous differential equation exactly; the code advances time in steps of $\Delta t = 0.01$ s. We need the exact discrete update for a single first-order lag $\tau\dot{y} + y = u$.

Step-by-step: Discretising one first-order lag

Step 1. The continuous solution of $\tau\dot{y} + y = u$ with u held constant over one sample interval is, by the standard first-order-lag solution formula,

$$y(t) = y(0)e^{-t/\tau} + u\big(1 - e^{-t/\tau}\big).$$

Step 2. Evaluate at $t = \Delta t$, writing $y[k]$ for $y(k\,\Delta t)$ and defining the **decay factor** $a = e^{-\Delta t/\tau}$:

$$y[k] = a\,y[k-1] + (1-a)\,u[k].$$

This is the update rule the code calls `cascade_exact`. (An older variant, `cascade`, used $u[k-1]$ instead of $u[k]$ on the right, introducing exactly one extra sample of pure delay per lag; that variant is kept only for Learner 1's originally-published numbers, and is *not* used for the bias analysis in §7, because that one extra sample of delay per stage would put an artificial floor under how small the measured bias can ever get.)

$$y[k] = a\,y[k-1] + (1-a)\,u[k], \qquad a = e^{-\Delta t/\tau} \tag{13}$$

Lay example: Numeric check of the decay factor

For $\tau = 1.0$ s and $\Delta t = 0.01$ s: $a = e^{-0.01/1.0} = e^{-0.01} \approx 0.99005$. So each 0.01 s step, the previous value is kept at 99% strength and the new input contributes the remaining 1%. For $\tau = 0.05$ s (the fastest mode): $a = e^{-0.01/0.05} = e^{-0.2} \approx 0.8187$, so 18% of the new input shows up immediately — a much quicker filter, as expected for a small τ.

Applying (13) three times in sequence, with the appropriate felt time constants from (12), gives the discrete simulation of the whole object (11).

5.4 Movement and exploration

Each trial's command signal has two parts:

$$u(t) = u_{\text{ref}}(t) + u_{\text{explore}}(t). \tag{14}$$

5.4.1 The reference reach: minimum jerk

u_{ref} is a smooth out-and-back reach built from the **minimum-jerk** profile, the standard model of natural human point-to-point movement (it is the trajectory that minimises the integral of squared

jerk, i.e. the smoothest possible movement between two points in fixed time):

$$s(t) = \mathrm{clip}\left(\frac{t}{t_{\mathrm{move}}}, 0, 1\right), \qquad u_{\mathrm{ref}}(t) = 10s^3 - 15s^4 + 6s^5. \tag{15}$$

Step-by-step: Where the coefficients $10, -15, 6$ come from

Step 1. We want a polynomial $p(s) = c_3 s^3 + c_4 s^4 + c_5 s^5$ (no constant, linear, or quadratic term) that satisfies $p(0) = 0$, $p(1) = 1$, and has zero velocity and zero acceleration at both endpoints ($p'(0) = p'(1) = 0$, $p''(0) = p''(1) = 0$): a movement that starts and ends completely smoothly.
Step 2. These four conditions on a three-coefficient polynomial (plus $p(1) = 1$) are exactly enough equations to solve for c_3, c_4, c_5 uniquely. Solving that linear system gives $c_3 = 10$, $c_4 = -15$, $c_5 = 6$, which is (15). (This is the same well-known result used throughout robotics and motor-control research for smooth point-to-point trajectories.)

The full reference is one such profile forward, followed by a mirrored profile backward, timed to start once the outward reach has mostly finished (`reach_reference`):

$$u_{\mathrm{ref}}(t) = \underbrace{u_{\mathrm{minjerk}}(t)}_{\text{out}} + \underbrace{-u_{\mathrm{minjerk}}\big(t - (t_{\mathrm{move}} + 0.35)\big)}_{\text{back}}.$$

5.4.2 Exploration: band-limited noise, not white noise

u_{explore} adds a small amount of extra variability so the learner has something to learn from beyond a single fixed trajectory. This is where a critical modelling decision is made.

Step-by-step: Why exploration cannot be white noise

Step 1. White noise contains equal energy at every frequency, including frequencies far above what any real muscle could produce. If we drove the object with white noise, we would excite the 10 rad/s and 20 rad/s transient poles just as strongly as the 1 rad/s dominant pole, on every single trial, regardless of stiffness or movement speed.
Step 2. If every trial already contains strong information about all three modes, there is no penalty at all for a learner that tries to identify all three modes from trial one, and the entire sample-complexity argument in this document (and in the meeting note) becomes untestable: there would be nothing for a curriculum to buy.
Step 3. Real voluntary movement is smooth. The only physically realistic way to put energy near the fast poles is to move quickly. The code therefore generates exploration by passing white noise $w[k]$ through *another* first-order lag with time constant $t_{\mathrm{move}}/3$, and then rescaling to unit standard deviation:

$$\tilde{y}[k] = a\,\tilde{y}[k-1] + (1-a)\,w[k], \qquad a = e^{-\Delta t/(t_{\mathrm{move}}/3)}, \qquad u_{\mathrm{explore}}[k] = \frac{\tilde{y}[k]}{\mathrm{std}(\tilde{y})}. \tag{16}$$

This produces a signal whose energy is concentrated below roughly $3/t_{\mathrm{move}}$ rad/s: faster reaches (*smaller* t_{move}) generate exploration with *more* high-frequency content, exactly matching the intuition that a faster movement is needed to reveal a faster mode.

Lay example: Band-limited exploration

A short, snappy movement ($t_{\mathrm{move}} = 0.25$ s) has meaningful energy up to about 12 rad/s — enough to excite the τ_1 mode (10 rad/s) but only weakly the τ_2 mode (20 rad/s). A slow, careful movement ($t_{\mathrm{move}} = 0.8$ s) only has energy up to about 3.75 rad/s — nowhere near either transient. This is why a curriculum that starts slow and speeds up is doing something physically meaningful, not an arbitrary schedule.

The felt output including sensor noise is:

$$y(t) = \text{cascade}_{\text{exact}}\big(\text{felt}(\tau, \xi),\, u(t)\big) + \sigma\, n(t), \qquad n(t) \sim \mathcal{N}(0, 1), \tag{17}$$

where σ (`NOISE_STD`) sets the sensor-noise scale.

6 Part IV — Trial failure (surprise)

Thrish, in the meeting: "If stabilisation is not there the ball goes out, the task fails, there is no learning." We now make this quantitative.

6.1 Defining surprise and tolerance

At every instant of a trial, the learner compares what it *predicted* it would feel, $\hat{y}(t)$, against what it *actually* feels, $y(t)$. The largest such mismatch anywhere in the trial is the **surprise**:

$$\text{surprise} = \max_t \big|y(t) - \hat{y}(t)\big|. \tag{18}$$

The amount of surprise a limb can absorb before something goes visibly wrong (the ball goes out of the cup) is the **tolerance**, and it is modelled as growing linearly with co-contraction:

$$\text{tolerance} = \text{S\_BASE}\,\frac{\xi}{\xi_{\text{soft}}}, \quad \xi_{\text{soft}} = 0.7, \quad \text{S\_BASE} = 0.15. \tag{19}$$

Step-by-step: What happens when the tolerance is crossed

Step 1. At every time sample k in the trial, compute $|y[k] - \hat{y}[k]|$ and compare it against the tolerance in (19).
Step 2. Find the *first* sample index k^* at which this quantity exceeds the tolerance. If no such sample exists, the trial **succeeds** and all n_t samples are usable.
Step 3. If k^* exists, the trial **fails** at that instant. Only the data from samples 0 through $k^* - 1$ (before the failure) are handed to the learner — data *after* a spill obviously cannot be trusted, but the data *before* it are still real physical measurements and are not thrown away, provided there are enough of them ($\geq$ MIN_USABLE $= 20$ samples) to be worth fitting.

Lay example: Failure, with numbers

Soft arm ($\xi = 0.7$): tolerance $= 0.15 \times (0.7/0.7) = 0.15$. A prediction error of just 0.15 units anywhere in the trial ends it early.
Stiff arm ($\xi = 3$): tolerance $= 0.15 \times (3/0.7) \approx 0.64$. The very same wrong model, making the very same mistake, now survives the whole trial, because the tolerance is more than four times larger.
So stiffness buys **safety while your model is still wrong**, not any kind of magic identification speed — that distinction turns out to be the crux of the whole study.

7 Part V — The bias law (central finding)

This is the most important derivation in the entire document, because it is the one calculation that finally explains *why* co-contraction should help a learner, in a way that survives scrutiny (unlike the sample-complexity argument discussed in §8, which does not survive scrutiny).

7.1 Low-frequency (Taylor) expansion of the true object

The idea: at low frequency (i.e. for slow, careful movements, or equivalently for small s near the origin), any transfer function can be approximated by its first few terms in a power series in s. We compute that series for the true object (11).

Step-by-step: Expanding $G(s)$ term by term

Step 1. Recall the geometric-series expansion, valid for small z:

$$\frac{1}{1+z} = 1 - z + z^2 - z^3 + \cdots$$

Apply this to each of the three factors of (11) with $z = \tau_d s$, $\tau_1 s$, $\tau_2 s$ respectively, keeping only up to the linear (first-order) term in s because we only need the leading behaviour:

$$\frac{1}{1+\tau_d s} \approx 1 - \tau_d s, \quad \frac{1}{1+\tau_1 s} \approx 1 - \tau_1 s, \quad \frac{1}{1+\tau_2 s} \approx 1 - \tau_2 s.$$

Step 2. Multiply the three approximations together, and again keep only terms up to linear order in s (any product of two "$-\tau s$" terms is $\mathcal{O}(s^2)$ and is dropped):

$$G(s) \approx (1-\tau_d s)(1-\tau_1 s)(1-\tau_2 s) = 1 - (\tau_d + \tau_1 + \tau_2)s + \mathcal{O}(s^2).$$

$$G(s) = 1 - (\tau_d + \tau_1 + \tau_2)s + \mathcal{O}(s^2). \tag{20}$$

7.2 Matching a simplified (first-order) model to the same expansion

Now suppose a learner, believing the object is only *first order*, fits a single lag $\hat{G}(s) = 1/(1+\hat{\tau}s)$. Expand this exactly the same way:

$$\hat{G}(s) \approx 1 - \hat{\tau}s + \mathcal{O}(s^2).$$

Step-by-step: Matching term by term to find the learner's bias

Step 1. A least-squares fit that matches the true low-frequency behaviour as closely as possible will match the two expansions (20) and the one above term by term, because the constant terms (both equal to 1, guaranteeing the right steady-state gain) already agree, and the next available term is the linear one.

Step 2. Setting the linear coefficients equal:

$$-\hat{\tau} = -(\tau_d + \tau_1 + \tau_2) \quad \Longrightarrow \quad \boxed{\hat{\tau} = \tau_d + \tau_1 + \tau_2}$$

$$\hat{\tau} = \tau_d + \tau_1 + \tau_2. \tag{21}$$

In plain language: **three lags in a row feel, at low frequency, exactly like one lag whose time constant is the sum of the three.** The reduced fit is not merely *noisy* around the true τ_d; it is *systematically shifted* by the neglected time constants:

$$\text{bias} = \hat{\tau}_d - \tau_d \approx \tau_1 + \tau_2 = 0.1 + 0.05 = 0.15\,\text{s}. \tag{22}$$

Lay example: Why this is different from "noise"

If the only problem were sensor noise, collecting more trials would average the noise away and the estimate would converge to the true $\tau_d = 1.0\,\text{s}$. That is exactly what happens for a learner that fits all three modes at once (Learner 1's setup, §8). But a learner that has decided, correctly or not, to use *only one* lag can never converge to the true τ_d no matter how many trials it runs, because (21) is what the best possible one-lag fit *is*. This systematic error is called a **misspecification bias**, and it is the phenomenon co-contraction acts on.

7.3 Effect of co-contraction on the bias

Under stiffening, the object is not what changes; what changes is that the *felt* fast lags are divided by $\rho(\xi)$, per (12). Substituting the felt time constants into (21) and (22) in place of the true ones (because it is the felt signal that any learner, biological or algorithmic, actually gets to see):

Step-by-step: Deriving the full bias law

Step 1. Replace τ_1, τ_2 in (21) by their felt values from (12), $\tau_1/\rho(\xi)$ and $\tau_2/\rho(\xi)$, while τ_d is left unchanged (it is not compressed by stiffening):

$$\hat{\tau}(\xi) = \tau_d + \frac{\tau_1}{\rho(\xi)} + \frac{\tau_2}{\rho(\xi)} = \tau_d + \frac{\tau_1 + \tau_2}{\rho(\xi)}.$$

Step 2. Subtract the true τ_d from both sides to isolate the bias:

$$\boxed{\hat{\tau}(\xi) = \tau_d + \frac{\tau_1 + \tau_2}{\rho(\xi)}, \qquad \text{bias}(\xi) = \frac{\tau_1 + \tau_2}{\rho(\xi)}} \tag{23}$$

This is the central equation of this document. **Co-contraction divides the misspecification bias by the exact same separation ratio $\rho(\xi)$ that governs pole separation in §4.** It is a statement purely about *bias*, and unlike variance, bias does not shrink with more data — it shrinks only by changing ξ.

Measured in `bias.py` (12 reaches per ξ, least squares fit)

The code (`fit_first_order`) simulates 12 reaches at each ξ, fits a single lag $\hat{\tau}$ to the resulting data by least squares (`minimize_scalar` over $\log \hat{\tau}$), and compares the fitted bias $\hat{\tau} - \tau_d$ against the closed-form prediction (23):

ξ	ρ	fitted bias (s)	predicted bias, $(\tau_1 + \tau_2)/\rho$ (s)
0.7	1.0	0.180	0.150
1.2	3.47	0.040	0.043
2.0	13.93	0.005	0.011
3.0	33.97	0.001	0.004
4.0	61.98	0.0006	0.0024

The fitted and predicted columns agree to within the residual noise and approximation error of the leading-order Taylor expansion at every ξ tested, across a bias that shrinks by more than **two orders of magnitude**.

Lay example: Bias law, in one sentence

You believe the cup has one time constant. With a soft arm you measure ≈ 1.18 s (close to the sum of all three true time constants). With a very stiff arm ($\xi = 4$) you measure ≈ 1.0006 s, almost exactly the true 1.0 s. Co-contraction did not change the cup at all — it stopped the fast, unmodelled parts of the cup's behaviour from leaking into, and polluting, your **simple** one-parameter estimate of the slow part.

8 Part VI — Learner 1 (gradient descent, order unlocked by hand)

8.1 What it believes, and why that is the problem

Learner 1 (code: `learner.py`, `experiment.py`) is a **gray-box** learner: it is told the correct model structure, three lags in series, and only has to find the right numbers $\hat{\tau}_d, \hat{\tau}_1, \hat{\tau}_2$ for them. Crucially,

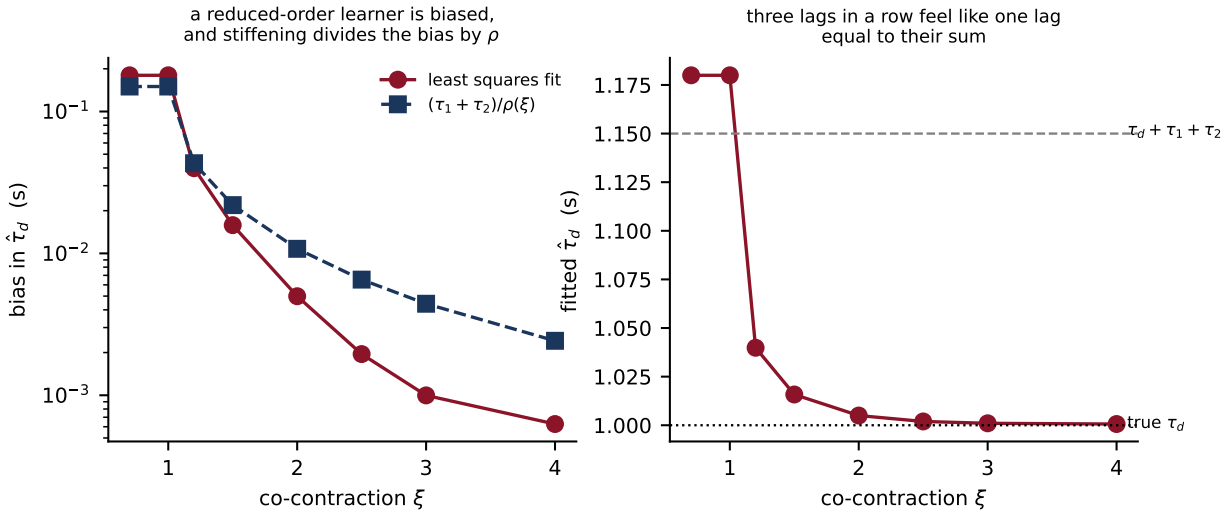

Figure 1: **Bias law, panel by panel.** *Left panel*: bias in $\hat{\tau}_d$ plotted against ξ on a log scale, comparing the actual least-squares fit (solid, circles) against the closed-form prediction $(\tau_1 + \tau_2)/\rho(\xi)$ (dashed, squares). Both curves fall together from about 0.18 s at $\xi = 0.7$ toward well under a millisecond by $\xi = 4$. The two curves tracking each other this closely across two decades of bias is the direct experimental confirmation of (23). *Right panel*: the fitted $\hat{\tau}_d$ itself (not the bias) plotted against ξ, together with two horizontal reference lines: the true $\tau_d = 1.0$ s (bottom) and the naive sum $\tau_d + \tau_1 + \tau_2 = 1.15$ s (top, the fully-soft prediction from (21)). The fitted curve starts near the top line at low ξ and descends smoothly toward the bottom line as ξ increases, visually showing the learner's belief moving from "systematically too slow" to "correct" purely as a function of stiffness.

when it is only allowed to use $n_{\text{unlocked}} \in \{1, 2, 3\}$ of the three parameters, the remaining ones are **forced to exactly zero** (a pure pass-through, no lag at all), rather than being allowed to settle on whatever value best explains the data. This single design choice, made before the bias law of §7 had been derived, is what prevents Learner 1 from ever exhibiting that bias law: when $n_{\text{unlocked}} = 3$ (which happens in every condition eventually, since Learner 1's curriculum always ends by unlocking all three parameters), the learner is **correctly specified**, meaning it uses exactly the true model structure, and a correctly specified least-squares fit converges to the true parameters as more data arrive, with an error that is pure random noise, shrinking as $1/\sqrt{N}$ in the number of trials N. There is no systematic bias left for co-contraction to remove, because none was ever introduced.

Initial guesses before any learning: $\hat{\tau} = (0.30, 0.03, 0.015)$ s — deliberately wrong, roughly three times too fast on every mode, so the learner has real work to do.

8.2 Prediction

Given the command u actually sent this trial and the stiffness ξ chosen for it, the learner's prediction is built the same way the true output is built in (17), but using the learner's *current beliefs* $\hat{\tau}$ instead of the truth:

$$\hat{y}(t) = \text{cascade}\big(\text{felt}(\hat{\tau}, \xi), u\big). \tag{24}$$

8.3 Learning rule: gradient descent

Plain-language guide: Gradient descent, from first principles

The goal. We have a cost function $\mathcal{C}(q)$ (here, half the sum of squared prediction errors, with $q = \log \hat{\tau}$ the parameters we control) and we want the q that makes $\mathcal{C}$ as small as possible.
The idea. The gradient $\nabla\mathcal{C}(q)$ is a vector that points in the direction in which $\mathcal{C}$ increases *fastest*. So $-\nabla\mathcal{C}(q)$ points in the direction of *steepest decrease*. Taking a small step in that direction,

and repeating, is guaranteed (for a small enough step size η) to reduce $\mathcal{C}$ on every step, and to converge to a point where the gradient is zero, i.e. a local minimum.
Why it can be slow. If the cost surface is shaped like a long, narrow valley (some directions in parameter space change the cost a lot, others barely at all — this is exactly the situation §8.5.1 below calls "ill-conditioned"), plain gradient descent zig-zags slowly along the valley floor instead of moving directly toward the minimum. This is the mathematical reason ill-conditioning matters for a gradient learner, and it is used explicitly in §8.5.1.

Concretely, the code keeps the residuals from the last 3 trials (`BUFFER=3`) stacked together, $r = y - \hat{y}$ (all trials concatenated), with cost $\mathcal{C}(q) = \frac{1}{2}\sum_t r_t^2$. Parameters are stored in **log** time constants, $q = \log \hat{\tau}$ (this keeps $\hat{\tau}$ automatically positive, and makes a fixed step size in q correspond to a fixed *percentage* change in $\hat{\tau}$, which is fairer across modes that differ by a factor of 20 in true magnitude).

Step-by-step: From cost to update rule

Step 1. By the chain rule, $\dfrac{\partial \mathcal{C}}{\partial q_k} = \sum_t r_t \dfrac{\partial r_t}{\partial q_k}$. Writing this as a matrix product with the **Jacobian** $J_{tk} = \partial r_t / \partial q_k$ (computed numerically by finite differences in the code, i.e. nudging each q_k by a tiny amount and seeing how much r changes):

$$\nabla \mathcal{C} = J^\top r.$$

Step 2. Take a gradient-descent step with learning rate $\eta = 3$, averaged over the m residual samples so the step size does not depend on how much data happens to be in the buffer:

$$q \leftarrow q - \eta \frac{J^\top r}{m}. \tag{25}$$

Step 3. Repeat (25) 12 times per trial (`STEPS_PER_TRIAL=12`), recomputing r and J at the updated q each time (this makes it closer to a proper Gauss–Newton-style fit within each trial, while still being called "gradient descent" because the underlying update direction is the gradient).

8.4 Conditions S1–S5

Code	Tuning schedule	What is imposed
S1	soft, fast, 3 modes unlocked from trial 1	the "deep end"
S2	$\xi : 3 \to 1.3 \to 0.7$, slow→fast, unlock $1 \to 2 \to 3$	the Thrish curriculum
S3	stiff throughout, 1 mode only, forever	never allowed to learn transients
S4	soft throughout, only speed staged, unlock $1 \to 2 \to 3$	tests the speed channel alone
S5	random ξ drawn fresh every trial	control for mere variability

The stage-advance rule for S2 and S4 (`_maybe_advance`) moves to the next stage once at least `MIN_TRIALS_PER_STAGE`=6 trials have been run *and* the prediction error has stopped improving for `PLATEAU_STREAK`=2 consecutive trials — i.e. the schedule advances when the current stage's learning has plateaued, not on a rigid trial count.

8.5 Results (20 seeds, 80 trials) and why they refute the speed claim

Learner 1 refuted the "curriculum learns faster" claim

	median trials to criterion	reached criterion	final error	failed trials
S1 deep end	47	100%	0.043	10.4
S2 curriculum	80	55%	0.105	0.0
S3 stiff forever	— (never)	0%	0.439	0.0
S4 bandwidth only	70	100%	0.084	11.55
S5 random	— (never)	0%	0.233	0.95

S2 needed a **median 33 trials more** than S1 to reach the same criterion (two-sided Mann–Whitney with Holm correction, $p \approx 1.6 \times 10^{-7}$). The curriculum won decisively on **safety** (zero failed trials versus 10.4 for S1), but lost on speed.

8.5.1 Two diagnostic facts that explain why

Both computed by `diagnose.py`, using the curvature (information) matrix $H^\top H$ evaluated at the true parameters (see §10 for the exact definition of κ).

1. **Co-contraction makes the full three-parameter fit harder, not easier, to identify.** The condition number $\kappa(\Phi)$ of the information matrix for the *complete* three-mode fit rises from about 5.5×10^3 when soft ($\xi = 0.7$) to about 1.8×10^{16} when stiff ($\xi = 3$) — an increase of **thirteen orders of magnitude**. Over the same range, the curvature on just the dominant mode's own parameter only improves from 2.54×10^{-2} to 2.72×10^{-2}: about 7%. Stiffening buys almost nothing for the direction that matters, while making the other two directions almost invisible to the data (this is exactly the analytic content of (23), seen from a different angle: the fast modes' information about themselves vanishes, but — for a learner that *must* fit them, as Learner 1 eventually must — vanishing information about a required parameter is bad, not good).
2. **The parameter directions that are hardest to learn are also the ones that matter least behaviourally.** Taking equal-sized steps along each eigenvector of the curvature matrix and measuring how much the resulting impulse response changes, the direction with the smallest curvature (i.e. slowest to learn, by a factor of about 5500× relative to the largest) changes behaviour by only about 28× less than the best-conditioned direction. In other words, exactly the directions the sample-complexity bound $N(r, \delta) \propto \sigma^2 r/(\delta \lambda_{\min}(\Phi_r))$ (see the meeting note, `meetings/2026-07-01-meeting-thrish.tex`) says are expensive to estimate precisely are the directions whose imprecision costs the least in terms of actual behaviour. This is why a bound on *parameter* error does not translate into a bound on *behavioural* error, and it is the single technical reason the "order reduction buys faster learning" argument, as originally stated, does not survive contact with an identification problem that is correctly specified.

9 Part VII — Learner 2 (two-timescale EKF + ARD)

Learner 2 (code: `kalman_learner.py`, `experiment2.py`) is built to remove both defects identified above:

1. **Misspecification.** Learner 1 always used the true three-lag structure whenever a mode was unlocked, so it could never exhibit the bias law (23). Learner 2 always carries *all three* modes, but lets each transient mode's *participation* in the prediction vary continuously between "fully present" and "not present", so it can genuinely behave as a lower-order model when the data do not support more.
2. **Overconfidence about the unobservable.** A fixed-step learner that receives essentially no information about a mode (because it is stiffened away) still takes fixed-size steps on that mode's parameter, driven purely by noise — a **random walk** that can destroy a perfectly good previous estimate. Learner 2 tracks its own **uncertainty** explicitly, so that a parameter direction with no information in the current data is simply left alone rather than perturbed.

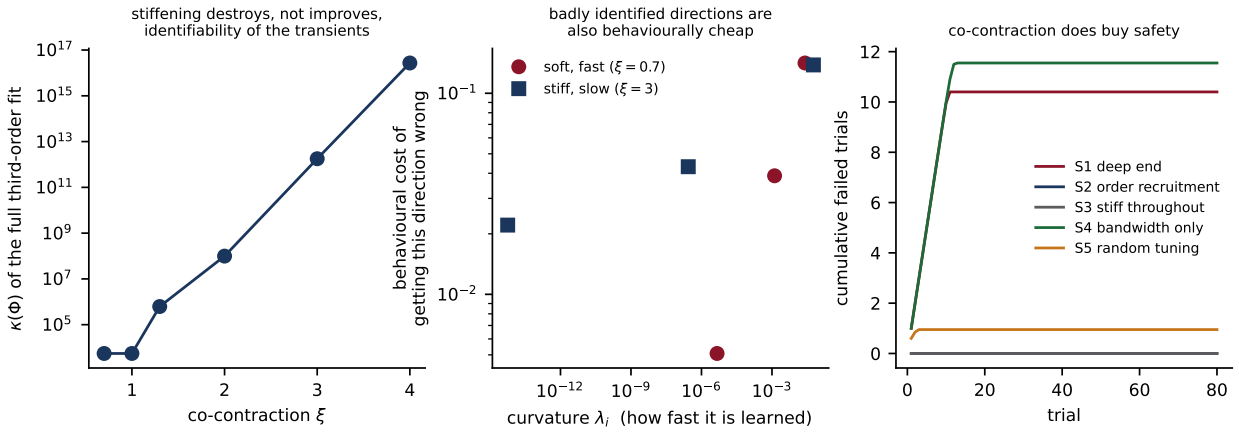

Figure 2: **Learner 1 diagnostics, panel by panel.** *Left panel*: $\kappa(\Phi)$ for the full three-parameter fit, plotted against ξ on a log scale. The line rises steeply and monotonically: stiffening steadily destroys the identifiability of the complete model, contrary to the naive expectation that a "simpler-looking" system should be easier to fit in full. *Centre panel*: for each eigen-direction of the curvature matrix, curvature (x-axis, how fast that direction is learned) plotted against behavioural cost (y-axis, how much the impulse response changes if that direction is wrong by a fixed amount), both on log scales, for two stiffness levels. The points line up along a rising diagonal trend: low curvature (slow to learn) systematically pairs with low behavioural cost (does not matter much), for both stiffness levels. *Right panel*: cumulative number of failed trials over the 80-trial run, one curve per condition. S1 and S4 climb quickly to roughly 10–12 failures and then flatten (once the model becomes good enough that the tolerance in (19) is no longer crossed); S2 and S3 stay at exactly zero throughout, because they never risk a soft, fast, badly-modelled reach.

9.1 State vector

$$q = \left[\log \hat{\tau}_d,\ \log \hat{\tau}_1,\ \log \hat{\tau}_2,\ w_1,\ w_2\right]^\top. \tag{26}$$

The first three entries are the same log-time-constants as Learner 1. The two new entries, $w_1, w_2 \in [0, 1.2]$, are **participation weights**: real numbers (not switches) describing how much of each transient mode is currently included in the learner's own prediction.

9.2 Blend dynamics: why not just set $\tau = 0$?

Rather than removing a mode by literally setting its time constant to zero (which is what "locking" a mode meant for Learner 1), each transient mode's lag is **blended** with a pass-through, in proportion w:

$$\text{blend}(\tau, w)[x] = (1 - w)\, x + w \cdot \text{lag}(\tau)[x]. \tag{27}$$

- At $w = 1$: blend $= \text{lag}(\tau)[x]$, the mode is fully present, exactly as in Learner 1.
- At $w = 0$: blend $= x$, the mode contributes nothing to the prediction, matching what "mode absent" should mean — *but* the formula still contains τ and w as adjustable quantities, so the *sensitivity* of the prediction to a small change in w (its partial derivative) is not zero even when w itself is zero.

Step-by-step: Checking that the derivative survives at $w = 0$

Differentiate (27) with respect to w:

$$\frac{\partial}{\partial w}\Big[(1 - w)x + w \cdot \text{lag}(\tau)[x]\Big] = -x + \text{lag}(\tau)[x] = \text{lag}(\tau)[x] - x.$$

This does not depend on the current value of w at all — it is the same nonzero quantity (as long as $\text{lag}(\tau)[x] \neq x$, i.e. the lag actually does something) whether w is currently 0, 0.5, or 1. So the

gradient-based filter always has *some* signal telling it whether to raise or lower w, even from a starting point of exactly zero.

Lay example: Volume knobs versus deleting a mode entirely

Wrong approach (what an earlier attempt at Learner 2 did, and which had to be fixed): encode "mode off" by setting $\hat{\tau}_2 = 0$ directly. Then the model's output literally does not depend on $\hat{\tau}_2$ at that point — its derivative is exactly zero — so once the fast mode is switched off while stiff, no future data, however clearly it shows the transient during a later soft trial, can ever switch $\hat{\tau}_2$ back on. Pruning becomes **irreversible**, which contradicts the very phenomenon (relaxing later to learn the transient) the simulation is meant to demonstrate.
Right approach (what the code actually does): encode "mode off" by setting the *weight* $w_2 = 0$ instead, leaving $\hat{\tau}_2$ itself free. The derivative with respect to w_2 stays alive at $w_2 = 0$ (as just shown), so when soft, fast data eventually arrive and strongly favour a nonzero fast mode, w_2 can rise again in response. Pruning is **reversible**.

The full prediction chain, applying (27) twice after the always-on dominant lag (code: `kalman_learner._h`):

$$\begin{aligned} x_0 &= \mathrm{lag}(\hat{\tau}_d)[u] \\ x_1 &= \mathrm{blend}\big(\hat{\tau}_1/\rho(\xi),\, w_1\big)[x_0] \\ \hat{y} &= \mathrm{blend}\big(\hat{\tau}_2/\rho(\xi),\, w_2\big)[x_1] \end{aligned} \tag{28}$$

9.3 Effective order

$$\boxed{n_{\text{eff}} = 1 + w_1 + w_2} \tag{29}$$

This is exactly the "effective order" quantity the meeting note makes its central observable, made concrete: it is simply the sum of the dominant mode (always fully present, contributing 1) plus however much of each transient mode the learner currently believes is real. It is never set by hand in the code; it is read off the current weights, which are themselves determined by the filter and the ARD mechanism below.

9.4 Extended Kalman Filter (EKF) — full derivation from Bayes' rule

Plain-language guide: Why a Kalman filter, from the ground up

The problem. We have a belief about the unknown parameters q, and each trial we get a noisy measurement that depends on q nonlinearly through (28). We want a principled way to (a) update the belief using the new data, and (b) keep track of *how confident* we should be, so a parameter with no information in the data is correctly left uncertain rather than being nudged around by noise.
The Bayesian starting point. Represent the belief as a Gaussian distribution over q: mean $\hat{q}$ (the current best guess) and covariance P (how spread out, i.e. uncertain, the belief is in every direction). Before seeing new data, let the belief drift by adding process noise Q (this models the idea that the true parameters, or at least our confidence about them, can change slightly from trial to trial):

$$P_{\text{pred}} = P + Q.$$

Combining a Gaussian prior with a Gaussian likelihood. If the prior on q is Gaussian with covariance P_{pred}, and the new measurement y is $y = h(q) + \text{noise}$ with noise variance R, then Bayes' rule for Gaussians says the posterior is Gaussian with **precision** (inverse covariance) equal to the sum of the prior's precision and the measurement's precision, weighted by how

sensitive h is to q (its Jacobian H):

$$P_{\text{post}}^{-1} = P_{\text{pred}}^{-1} + \frac{H^\top H}{R}.$$

This is the core Kalman-filter idea, and it is exactly analogous to how combining two independent noisy measurements of the same quantity gives a combined estimate with smaller variance than either alone: precisions simply add.

Nonlinearity. Because (28) is nonlinear in q (it passes u through exponentials of q), we cannot apply the Gaussian update exactly. The **Extended** Kalman Filter (EKF) approximates the nonlinear function by its **first-order Taylor expansion** (i.e. its tangent plane) around the current best guess, computes the update as if that linear approximation were exact, and then — because a single linearisation around a possibly-still-wrong guess can itself introduce error — the code repeats this linearise-and-update step three times per trial (an **iterated** EKF), re-linearising around the improved guess each time.

9.4.1 Setting up the linearisation

Write $h(q) =$ (28) evaluated at parameters q, for the whole trial's worth of samples at once, so $h(q)$ is a vector of predicted samples. The measurement error and its sensitivity to q are:

$$e = y - h(q), \qquad H = \left.\frac{\partial h}{\partial q}\right|_q. \tag{30}$$

H is computed numerically in the code by finite differences: nudge each entry of q by a tiny amount 10^{-3}, re-run h, and divide the change in output by the nudge size.

9.4.2 The information-form update, derived step by step

Step-by-step: From the Bayesian combination rule to the code

Step 1 (precision of the posterior). As derived above, combine the predicted precision P_{pred}^{-1}, the measurement precision $H^\top H/R$, and — because the participation weights carry an extra prior pulling them toward a target value (the ARD prior of §9.5) — an additional term $\text{diag}(\alpha)$ representing that extra prior's precision:

$$A = P_{\text{pred}}^{-1} + \frac{H^\top H}{R} + \text{diag}(\alpha). \tag{31}$$

Step 2 (posterior covariance). Invert to get the new covariance:

$$P_{\text{post}} = A^{-1}. \tag{32}$$

Step 3 (posterior mean). The posterior mean shift is the posterior covariance times the sum of all the "pulls" on q: the pull from the measurement error ($H^\top e/R$, "move toward what explains the data"), the pull from the ARD prior ($-\alpha(q - q_{\text{target}})$, "move toward the prior's preferred value, in proportion to how strongly the prior believes in it"), and the pull from the process-noise prior linking back to the previous belief ($-P_{\text{pred}}^{-1}(q - q_{\text{pred}})$, "don't drift arbitrarily far from where you started this trial"):

$$q \leftarrow q + P_{\text{post}} \left(\frac{H^\top e}{R} - \alpha(q - q_{\text{target}}) - P_{\text{pred}}^{-1}(q - q_{\text{pred}}) \right). \tag{33}$$

Step 4 (why this is the same as the textbook Kalman gain form). Equations (31)–(33) are the "information form" of the Kalman update. It is mathematically equivalent to the more commonly seen form written with an explicit "Kalman gain" matrix $K_{\text{gain}} = P_{\text{pred}} H^\top (H P_{\text{pred}} H^\top + R)^{-1}$ and update $q \leftarrow q_{\text{pred}} + K_{\text{gain}} e$; the information form is used here because it lets the extra ARD precision term $\text{diag}(\alpha)$ be added in directly, without needing a separate "pseudo-measurement" trick.

9.4.3 Why R is inflated during early trials

If R (the measurement noise variance) were fixed at the true sensor-noise level, the filter would treat *all* of the residual e as informative about q, even in early trials when most of the residual is actually caused by the model still being wrong, not by sensor noise. The code instead sets

$$R = \max\big(R_{\text{std}}^2,\ \text{mean}(e^2)\big),$$

so that while the fit is still poor, R is large (the filter treats the measurement as unreliable and updates cautiously); once the residual shrinks close to the true noise floor, R settles back to R_{std}^2 and the filter becomes appropriately confident.

Lay example: EKF, worked through one trial

Day 1: the learner's guess for the cup's main time constant is 0.3 s, and it is very unsure (P_{11} large). It makes a slow, stiff reach and feels a sluggish response. The prediction error e is large and mostly explained by $\hat{\tau}_d$ being too small, so $H^\top e/R$ points strongly in the direction of increasing $\hat{\tau}_d$; the update in (33) moves the estimate toward roughly 0.8 s and (32) shrinks the uncertainty P_{11} accordingly. Meanwhile, because the reach was stiff, the columns of H corresponding to w_1, w_2 are nearly zero (the fast modes contributed almost nothing to $\hat{y}$, per (12)), so the corresponding rows and columns of $H^\top H/R$ are nearly zero too, and those uncertainties barely shrink. The learner correctly "knows that it does not yet know" about the transients.

9.5 ARD — Automatic Relevance Determination, derived

Plain-language guide: ARD, from first principles

The idea. We want each participation weight w_i to be pulled toward zero ("this mode probably does not matter") *unless* the data consistently push back against that pull. This is achieved with a Gaussian prior on w_i centred at 0 with precision α_i: the larger α_i, the more strongly w_i is pulled toward zero, and the smaller its allowed range of movement.
The trick: let the data set α_i itself. Rather than fixing α_i by hand, ARD updates it after every trial based on how well determined w_i currently is. Define $\gamma_i = 1 - \alpha_i P_{ii}$, a quantity between 0 and 1 that measures how much of w_i's current uncertainty P_{ii} is explained by *data* rather than by the prior alone (when the prior dominates, $\alpha_i P_{ii} \to 1$ so $\gamma_i \to 0$; when data dominate, $\alpha_i P_{ii} \to 0$ so $\gamma_i \to 1$). Then update:

$$\alpha_i \leftarrow \frac{\gamma_i}{\max(w_i^2, \varepsilon)}. \tag{34}$$

Why this produces automatic pruning. If the data do not support mode i (its estimate w_i stays small and uncertain), γ_i stays away from 1 while w_i^2 in the denominator is small, so α_i does not blow up uncontrollably (it is capped in the code, `ALPHA_BOUNDS`, precisely so that pruning stays reversible rather than permanent) but stays large enough to keep pulling w_i toward zero. If instead the data strongly support a sizeable w_i, then w_i^2 in the denominator grows, driving α_i down and *releasing* the pull, letting w_i settle wherever the data actually want it.

This is precisely how n_{eff} in (29) falls toward 1 while stiff (the transients have no data to justify keeping their weights up, so ARD shrinks them) and rises again once co-contraction relaxes and soft, fast trials start to supply real information about the transients — without a single line of code anywhere saying "unlock mode 2 now".

9.6 Two timescales

Process noise on the state (`PROCESS_NOISE`, roughly):

$$Q = \mathrm{diag}\big(10^{-5},\ 2\times10^{-3},\ 2\times10^{-3},\ 2\times10^{-3},\ 2\times10^{-3}\big).$$

The dominant log-time-constant is allowed to drift 200 times more slowly than everything else. This is the concrete implementation of the "two-timescale" idea: a slow, well-retained channel for the part of the model that matters across the whole learning process, and a fast, more forgetful channel for the parts that are only relevant once they have been exposed by relaxing.

9.7 Conditions K1–K5

Identical tuning schedules to S1–S5, but with two crucial differences from Learner 1's setup:

- **No hand-imposed unlocking.** All three $\hat{\tau}$'s and both weights w_1, w_2 exist from trial one in every condition, including K3 (stiff throughout). Whatever effective order K3 ends up at is discovered by the filter, not decided in advance by the code.
- **Confidence-gated relaxation for K2 and K4.** Rather than advancing on a fixed trial count, the schedule advances once the posterior standard deviation of $\log \hat{\tau}_d$ falls below a threshold (`CONFIDENCE_SD`=0.04): the learner relaxes only once it is genuinely confident about the part of the model it can currently see, which is the direct computational version of Thrish's "learn that, then relax a little bit more".

9.8 Results (20 seeds, 60 trials) and what each number means

Learner 2 confirms the mechanism; still does not confirm the speed claim

	trials to criterion	final error	peak $\lvert\hat{\tau}_d - \tau_d\rvert$	final n_{eff}	failed trials
K1 deep end	5	0.031	0.691 s	2.18	4.0
K2 confidence-gated	19	0.030	0.087 s	2.18	0.0
K3 stiff forever	— (never)	0.404	0.013 s	1.00	0.0
K4 bandwidth only	4	0.030	0.026 s	2.19	1.1
K5 random	35	0.076	0.040 s	1.92	0.7

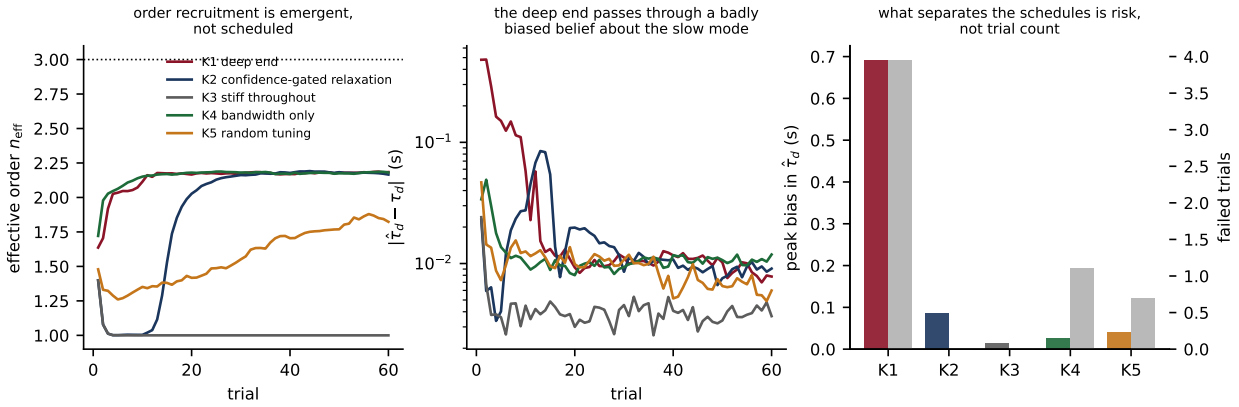

Figure 3: **Learner 2, panel by panel.** *Left panel*: n_{eff} ((29)) plotted against trial number for all five conditions. K3 (stiff throughout) drops to and holds exactly at $n_{\text{eff}} = 1.00$ for the entire run: the learner has discovered, purely from ARD shrinking w_1 and w_2 toward zero, that it should behave as a first-order system, with no rule anywhere telling it to. K2 tracks K3 closely at $n_{\text{eff}} \approx 1$ while stiff, then rises sharply to about 2.2 at the exact trial where the confidence-gated schedule permits relaxation (§sect. on K1–K5 above): this is progressive order recruitment happening as a consequence of the ARD mechanism responding to newly available data, not because a schedule told it to. K1 and K4, which never restrict themselves at all, rise to $n_{\text{eff}} \approx 2.2$ almost immediately. *Centre panel*: the absolute error $|\hat{\tau}_d - \tau_d|$ over trials, log scale. K1's curve starts at roughly 0.1 and briefly rises to nearly 0.7 before falling — this transient overshoot, entirely absent from K3, is the misspecification bias of (23) being incurred and then paid off as the learner is exposed to all three modes simultaneously while still uncertain. K2's curve stays low throughout the stiff stage and only shows a much smaller bump around the trial where it relaxes. *Right panel*: a bar chart with two axes, peak bias (coloured bars, left axis) and number of failed trials (grey bars, right axis), one pair per condition. K1 has both the largest peak bias and the most failures; K2 and K3 both have essentially zero failures, with K2's peak bias (from the same brief transient period visible in the centre panel) still roughly 8 times smaller than K1's. This panel is the clearest single visual statement of the paper's actual finding: **the schedules differ in the risk incurred while learning, not in how many trials learning takes.**

Lay example: Reading K2 versus K1, side by side

Both K1 and K2 end up at essentially the same final model (error ≈ 0.03, $n_{\text{eff}} \approx 2.2$). K1 gets there in only about 5 trials, but along the way its belief about the dominant time constant is, at its worst point, wrong by about 0.7 s — nearly as large as the true value itself — and 4 of its trials fail outright (the "ball goes out"). K2 takes roughly four times as many trials (about 19) to reach the same final model, but its worst-case error in the dominant time constant is only

about 0.09 s, and it never once fails a trial. In other words: **the curriculum buys a much safer route to the same destination, not a shorter one.**

10 Part VIII — Scoring metrics (every formula, fully explained)

Relative impulse error — the headline measure of "how wrong is the learned object model, overall?" Simulate an impulse (a very brief, very strong pulse) through both the true object and the learner's current belief about the object, and compare the two resulting responses:

$$\text{err} = \frac{\|y_{\text{imp}} - \hat{y}_{\text{imp}}\|_2}{\|y_{\text{imp}}\|_2}, \qquad \|v\|_2 = \sqrt{\textstyle\sum_k v_k^2}. \tag{35}$$

This is zero only if the learner's belief produces *exactly* the same impulse response as the truth at every time sample, and it is normalised by the size of the true response so that a value of, say, 0.1 means "the model's error is about 10% of the size of the true signal", regardless of the absolute scale of the signal.

Trials to criterion — the first trial number at which err < 0.10 for 3 consecutive trials in a row (a "streak", so a single lucky trial does not count as having converged).

Condition number $\kappa(\Phi)$ (Learner 1 diagnostics, computed by `diagnose.py`). Form the curvature (information) matrix $\Phi = H^\top H$ from the Jacobian H of the prediction with respect to the (log) parameters, evaluated at the true parameter values. This matrix has three eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$ (since Φ is symmetric positive semi-definite), each with an associated eigenvector telling you a direction in parameter space. The condition number is the ratio of the largest to the smallest:

$$\kappa(\Phi) = \frac{\lambda_1}{\lambda_3}. \tag{36}$$

Step-by-step: Why a large κ means "some directions are invisible"

Step 1. The eigenvalue λ_i measures how much the sum of squared prediction errors changes if you move a unit distance along eigen-direction i: a large λ_i means moving that way changes the predictions a lot (easy to detect from data), a small λ_i means moving that way changes the predictions very little (hard to detect).
Step 2. If λ_1/λ_3 is enormous (as it is for the stiff conditions, $\sim 10^{16}$), then direction 3 changes the predictions by a factor of 10^{16} less than direction 1 does. Any realistic amount of measurement noise will completely swamp whatever tiny signal exists about direction 3, making that parameter combination effectively unidentifiable from the available data.

11 Part IX — Conclusions and discussion

11.1 What we conclude

1. **Co-contraction manufactures time-scale separation** through the exact closed form $\rho(\xi) = (\xi + \sqrt{\xi^2 - 1})^2$ (9), derived directly from the mass–spring–damper physics (1)–(10). This part of the mechanism is uncontroversial physics, not a modelling choice, and both learners implement it identically.

2. **The precise, provable benefit for a low-order learner is bias reduction** (23), not the generic "fewer parameters therefore fewer trials needed" argument originally proposed. Stiffening divides the *misspecification bias* of a reduced-order fit by $\rho(\xi)$. This is a statement about a *systematic* error, not a random one, so unlike the sample-complexity argument it does not require appealing to how a learner's optimisation algorithm behaves — it is true of the *best possible* low-order fit, and it is measurable directly in people by relating a participant's fitted dominant time constant to their measured EMG co-contraction level.

3. **Progressive order recruitment can emerge from data alone** (Learner 2, condition K2): n_{eff} holds at almost exactly 1 while stiff and rises to about 2.2 precisely when co-contraction is relaxed, with no rule in the code ever specifying how many modes to use at a given time — it is a consequence of the ARD mechanism (§9.5) responding to which directions the data currently support.

4. **Staying stiff forever leaves a learner provably stuck at first order** (condition K3): n_{eff} settles at exactly 1.00, the dominant estimate $\hat{\tau}_d$ is essentially unbiased (error +0.0014 s), and the overall model error plateaus around 0.40 and never improves. This is discovered by the learner, not imposed by the code, which is the crucial methodological improvement over Learner 1's version of the same claim.

5. **The curriculum reduces risk, not trial count**, in every simulation run so far: S2 and K2 both have zero failed trials against 10–12 for the corresponding deep-end schedules S1/K1, while taking *more* trials, not fewer, to reach the same final model quality.

6. **A person starting stiff and relaxing later is rational** if their objective is something like "do not lose attempts to catastrophic failure while building a coarse but useful model of a new object", rather than "minimise the number of attempts needed to fully identify a correctly specified model as fast as statistically possible". The two objectives favour opposite strategies in these simulations, and the second objective is not, on reflection, a plausible description of how a person actually learns to handle a new object.

11.2 What we do *not* conclude

Do not claim these in the proposal — both simulations refute them

- "S2/K2 reaches criterion in fewer trials than S1/K1" — **false** in both learners, on every metric reported here. If anything the opposite is true.
- "The sample-complexity inequality $N(r, \delta) \propto \sigma^2 r/(\delta \lambda_{\min}(\Phi_r))$ implies faster behavioural learning under a curriculum" — refuted specifically for Learner 1 by the diagnostic in §8.5.1: ill-conditioned parameter directions are also behaviourally cheap directions, so a bound on parameter error is not a bound on behavioural error.
- "Bandwidth staging alone (condition K4, speed changes without any stiffness change) is worse than the full stiffness-plus-speed curriculum (K2)" — in Learner 2, K4 is in fact among the *fastest* schedules, reaching criterion in a median of only 4 trials.
- "A learner will always eventually recruit all three true modes" — n_{eff} asymptotes near 2.2, not 3, in every condition that reaches a final answer, because under the band-limited exploration actually used ((16)), the third (fastest) mode very rarely produces enough signal to be worth keeping according to the ARD criterion. This is the ARD mechanism working correctly, not a bug.

11.3 Recommended claim for the URF four-pager

A learner that treats the coupled hand-object system as lower order than it truly is inherits a systematic bias set by the neglected transient modes. Co-contraction divides that bias by the exact separation ratio $\rho(\xi)$ derived from the coupled mass-spring-damper dynamics, yielding an unbiased estimate of the dominant mode while the effective order of the internal model is temporarily suppressed. A learner that carries its own uncertainty over how many modes are currently supported by the data exhibits progressive order recruitment exactly when co-contraction is relaxed, reaching the same asymptotic model of the object as an unstiffened learner, but with substantially lower peak error in the dominant mode along the way and no failed attempts.

11.4 Open next steps (not yet simulated)

- **Close the loop.** Both learners identify a plant; neither controls one. A controller designed from the current posterior belief, with co-contraction setting its gain margin, is the natural next condition, and it would let instability during learning enter as part of the phenomenon being studied rather than being engineered away.
- **Sweep the cost of a failed trial.** Currently a failed trial simply loses the remainder of that trial's data. If recovering from a failure is expensive (as it plausibly is for a person, e.g. retrieving a dropped object), the ordering between K1 and K2 on trial count, not just on risk, might reverse.
- **Test the bias law directly against WP1 data.** Fit each participant's behaviour with a deliberately reduced-order model and check whether the residual bias in their fitted dominant time constant tracks $1/\rho(\hat{\xi})$ computed from their own measured electromyographic co-contraction.

12 Appendix — Code map

File	Role
`order_reduction/plant.py`	Object (11), $\rho(\xi)$ (9), cascades (13), reach (15), exploration (16)
`order_reduction/learner.py`	Learner 1: gradient descent (25)
`order_reduction/experiment.py`	S1–S5 trial loop, failure rule (18)–(19)
`order_reduction/diagnose.py`	$\kappa(\Phi)$ (36) and behavioural cost
`order_reduction/bias.py`	Bias law (23) verification
`order_reduction/kalman_learner.py`	Learner 2: EKF (31)–(33) + ARD (34)
`order_reduction/experiment2.py`	K1–K5 trial loop
`scripts/run_curriculum.sh`	Run Learner 1 figures (Figure 2)
`scripts/run_learner2.sh`	Run Learner 2 figures (Figures 1, 3)

Document generated to accompany `order-reduction-sim`*. Recompile after code changes; re-run simulations to refresh figures in* `outputs/curriculum/` *and* `outputs/learner2/`*.*